Addition 1 Resolution + Addition 2: Learning-Curve Experiment

Addition 1 resolution: hardware-region distance as a standalone signal

Attempt 1: hw-region-only gate inside the full online simulation

Re-ran the Addition 1 simulation with confidence collapsed to `hw_region_component` alone (dropping interval and query-support entirely), same seed, same stream. Result: **293/294 rows predicted (99.7%)** — the gate abstained on essentially nothing. Predicted-subset MAPE 40.50% vs. blind-all 40.99%: technically the right direction, but the effect size is negligible (one excluded row). This is because in the dynamic simulation, `seen_machines` grows the moment *any* row touching a new machine is measured — with only 2 new machines in a 5-machine universe, the signal gets neutralized after just 1–2 measurements and the rest of the 294-row stream is trivially "familiar" for the rest of the run. **This is not a clean demonstration** — it's confounded by the online loop's own bookkeeping.

Attempt 2: static, isolated validation — the clean positive result

To test the underlying claim without that confound: fit the p50 model once on the original 3-machine pool (c7i, m5a, r5n; no retraining), compute `hw_region_component` for all 294 streamed rows using a **fixed** reference set (just the original 3 machines, never updated), and check whether it separates actual prediction error.

group	n	mean abs. % error
LOW confidence (hw_region < 0.6)	84	**75.31%**
HIGH confidence (hw_region ≥ 0.6)	210	**27.26%**
blind-all (no gate)	294	40.99%

Correlation(hw_region_component, actual error) = −0.545. Strongly negative, exactly the intended direction.

Per-machine breakdown (by `prod_machine`, the side the component scores):

prod_machine	hw_region_component	n	mean abs. % error
c5a	0.419	84	**75.3%**
z1d	0.808	84	23.4%
c7i	1.000	42	21.6%
m5a	1.000	42	33.4%
r5n	1.000	42	34.5%

This is the clean positive result. Deploying to c5a — flagged as hardware-novel *before ever seeing a single c5a measurement* — carries more than 3x the error of deploying to z1d, which the signal correctly scored as safe. The HIGH-confidence group alone (27.26% MAPE) comfortably beats blind prediction

over everything (40.99% MAPE). This validates the Addition 1 finding as a standalone claim, decoupled from the online loop's dynamics: **hardware-signature distance to the nearest machine with training data is a working, validated abstention signal for cross-environment prediction**, distinct from and more reliable than prediction-interval width at this sample size.

Artifacts: `results/online_predictor_hwonly_log.csv`, `results/online_predictor_hwonly_summary.json`, `results/hw_region_static_validation.csv`.

Addition 2: Learning-curve experiment

Script: `learning_curve.py`. Figure: `results/learning_curve.png`.

Machines-seen axis (the money figure)

Training definition: for k seen machines, training rows are all pairs whose **dev machine** is one of the k seen machines (prod ranges freely over all 5, since prod only ever contributes a static hardware ratio — never leaked runtime data). Test rows: all pairs whose dev machine is *not* one of the k seen machines. Averaged over all `C(5,k)` combinations (5, 10, 10, 5 combos for k=1..4).

k	n train	n test	combos	learned mean MAPE	learned range	naive-linear mean MAPE	naive-linear range
1	84	336	5	40.13%	26.96–83.33%	23.03%	21.58–23.65%
2	168	252	10	30.54%	24.42–41.10%	23.03%	21.20–24.54%
3	252	168	10	29.43%	20.51–38.06%	23.03%	20.75–25.76%
4	336	84	5	27.99%	15.63–38.21%	23.03%	20.51–28.80%

MAPE drops from 40.13% to 27.99% as k rises from 1 to 4 — a clear, monotonic improvement. The gap to naive-linear narrows from **+17.10pp at k=1 to +4.96pp at k=4**. The online-learning premise is validated on this axis: seeing more distinct machines genuinely improves generalization to unseen ones. The k=1 point carries a very wide range (27–83%) — only 5 combinations exist at that k, and single-machine training is inherently high-variance; treat it as indicative, not precise.

The gap narrows substantially but does not close — even with 4 of 5 machines seen, the learned model is still ~5pp worse than the trivial single-ratio baseline. With only 5 machines total, k=4 is the ceiling this dataset can test; whether the trend would cross over with more hardware diversity is not something this data can answer, and we don't claim it would.

Data-density axis (secondary)

Fixed leave-one-machine-out split (4 seen, 1 held out, matching Phase 6/7 exactly), varying the fraction of the 21 queries used in training, tested on all queries against the held-out machine, averaged over the 5 possible held-out machines.

query fraction	n queries	learned mean MAPE	learned range	naive-linear mean MAPE
25%	5	33.32%	20.18–56.91%	23.03%
50%	10	30.95%	18.42–53.86%	23.03%
75%	16	31.46%	17.84–56.67%	23.03%
100%	21	31.26%	15.79–55.54%	23.03%

This curve is essentially flat after 50% of queries (30.95% → 31.46% → 31.26%, non-monotonic within noise) — going from 5 to 21 training queries per machine buys almost nothing (~2pp total, and most of that gap closes by 50%). Naive-linear is flat by construction (it doesn't use queries as training data at all).

Plain-language read

- **Does accuracy improve with more machines? Yes, clearly** — MAPE falls 40.1% → 28.0% from k=1 to k=4, a real and fairly steady trend, not noise (even accounting for wide per-combo spread at low k).
- **Does accuracy improve with more queries? Only marginally, and it saturates fast** — most of the (small) benefit is captured by 50% of the queries; the remaining half buys almost nothing. Query density is not the bottleneck.
- **Does the learned model close the gap to naive-linear with more data, or stay flat? It closes substantially (17.1pp → 5.0pp) but does not fully close** within the 5-machine ceiling this dataset offers. This directly answers the question Phase 7 left open: the gap to the simple baseline is *not* a fixed ceiling set by the feature set alone — it shrinks with more machine-hardware diversity — but 5 machines isn't enough data for the learned model to actually overtake naive-linear here. **This reframes, rather than overturns, Phase 7's conclusion**: naive-linear remains the right choice to deploy today given the data actually available, but the trend line suggests a learned approach could earn its complexity with materially more hardware diversity than this project collected — a concrete, falsifiable prediction for future work rather than a vague "more data would probably help."

Artifacts

- `learning_curve.py` — full experiment script.
- `results/learning_curve_machines_seen.csv`, `results/learning_curve_data_density.csv` — raw tables.
- `results/learning_curve.png` — the publishable figure (two panels, mean line + min/max band, both approaches on shared axes).